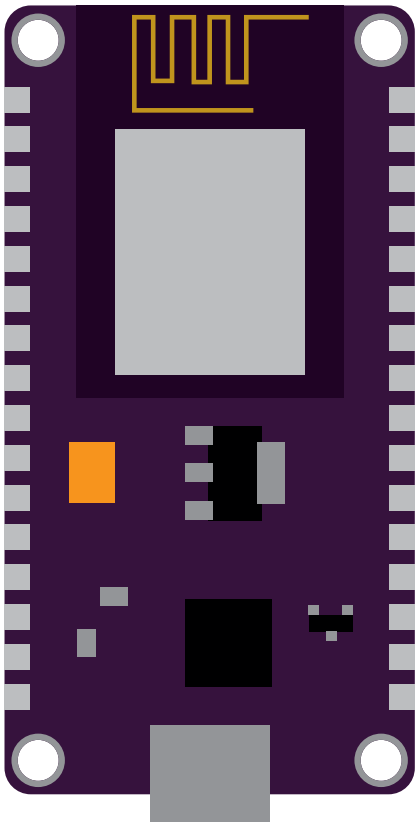
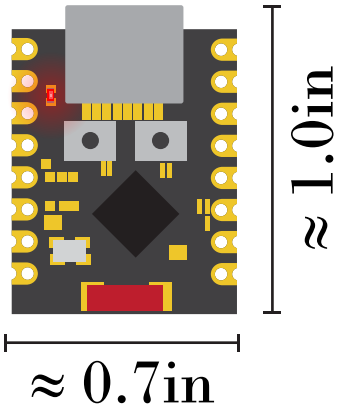
ESP8266



ESP32-C3 SuperMini



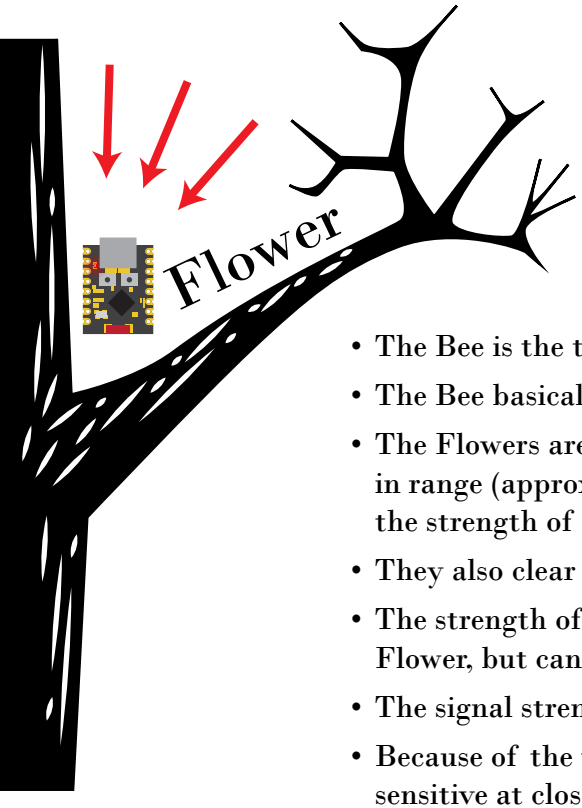
The ESP8266 and the 32 are both WiFi band radio chips. They can do a lot of different WiFi things, but we're just using the radios to transmit and receive a simple signal.

The 32 is the successor to the 8266 and improved in most ways. We're mostly using 32s, but I have some 8266s, and that's what I started the project with, so they're somewhat relevant.

They're most easily powered via USB, but anything 3.3-5V will do. They can output 3.3V through their GPIO (General Purpose In/Out) pins which can be programmed with the Arduino IDE software. The 32 has 13 GPIO pins, 10 of which are easy to use. The 8266 has 9 GPIO pins, 5 of which are easy.

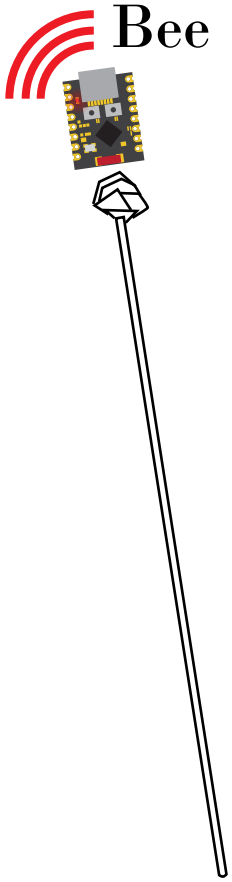
That output can be... Pretty much anything, sometimes with a little support hardware. Examples include: logic HIGH; logic LOW; PWM; simulated waveforms like sine or sawtooth; sampled audio; RGB LED control signals; LED panel control; LCD panel signals; most types of A/V signals... Even if you can't output something directly, there is usually some sort of work-around. The options are overwhelming. I'll get into it further along in this document.

beeART

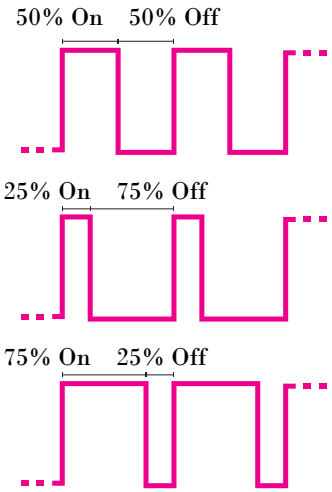


How Do?

- The Bee is the transmitter and the Flower is the receiver.
- The Bee basically shouts its name about every 25ms, or 40 times/second.
- The Flowers are listening to the Bees and keep a record of which Bees are in range (approx. 150ft, maybe more or less depending on obstacles), and the strength of the Bee's received signal.
- They also clear the record regularly so they can forget Bees out of range.
- The strength of a Bee's signal gives a general idea of its proximity to the Flower, but can also fluctuate with obstacles between the Bee and Flower.
- The signal strength is called the RSSI.
- Because of the way radio signals propagate, the RSSI is much more sensitive at close distances, and becomes less sensitive as the distance between the Bee and Flower grows.
- We can program the Flowers to react to different Bee names and signal strengths.
- Examples of the reactions include outputting various signals on the GPIO pins, and modulating existing output signals.
- Complex interactions are very possible.
- Each Flower can be programmed independently.
- Bees can be set up to receive signals as well, but that could increase the complexity of the project pretty significantly.
- For our purposes there is no real upper limit on the number of possible Bee names. Multiple Bees can also share the same name.
- It is also possible to add other sensors and things like buttons and knobs to both the Flowers and Bees. I'd rather not, for now, but we can keep it in our back pocket as an option.



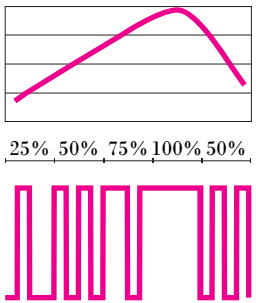
beeART



PWM stands for pulse width modulated. It's a common way to control things that only really turn on or off. The idea is that if you switch something on and off fast enough, and with some precision, you can simulate fractional power.

Consider a light bulb that can only be off or on. If we want it to shine at half brightness flicking it on and off by hand will be obvious, but doing it 1000 times a second will be much less obvious. If we want the bulb to shine at 25% brightness, then we adjust the duty cycle, or on time, so that the bulb is only lit for 25% of the time. Same idea if we want it brighter.

PWM is a common control method for motor speed, dimming lights, and heating elements. Usually the signal doesn't actually drive the load, but it drives the output of a transistor or something that allows the small signal to control a much larger amount of power.

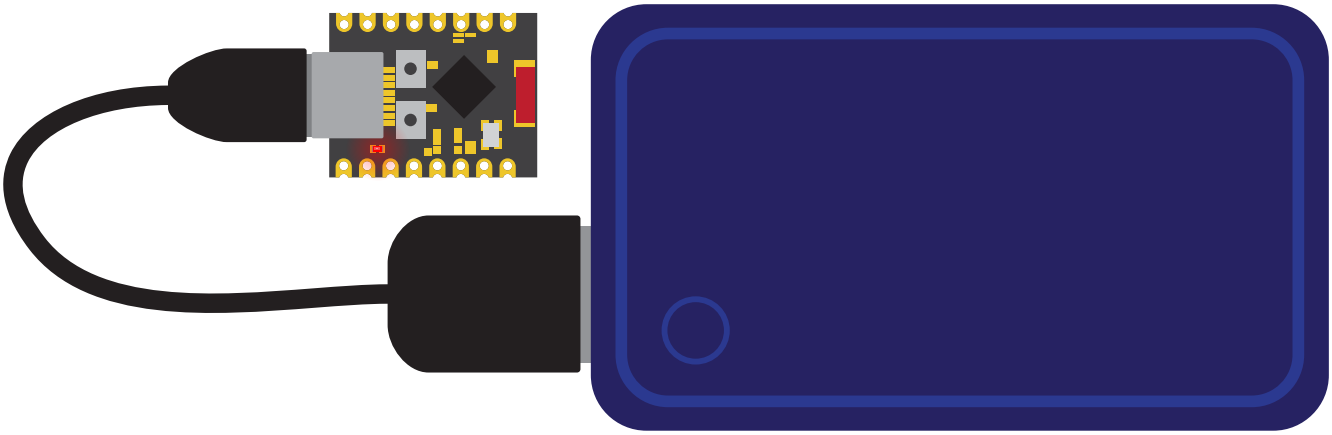


PWM can even be used for audio signals. There's techniques to smooth the PWM from a bunch of ons and offs into its average voltage. If that average voltage varies at audible frequencies, baby you got audio. It's the principal behind all digital audio and the reason why people say listening to MP3s sucks. The circuits that smooth the signal are called DACs, digital to analog converters. If we're doing any kind of audio output, aside from square waves, we'll probably want a DAC. There are DIY options, but the real thing is pretty cheap. The DIY options don't sound great, but it might also be what we're into for this project.

Beyond that even, there are a ton of different libraries that allow the ESPs to output just about anything you can think of. On the first page I mentioned RGB LEDs, LED panels, LCD panels, and most types of A/V signals, but that's just a few examples. I cannot overstate just how many different types of things can be controlled with an ESP.

We can ignore the details, but any device that says it can be controlled via I2C, SPI, or UART is compatible. Just about anything made for Arduinos is also compatible. Check out Adafruit or Sparkfun and see if anything catches your eye.

beeART



There are some limitations to the system. We'll need batteries or power banks for everything. Batteries are cheaper (something like a LiPo 18650) but we'll need to regulate them which is just another little cost and some time. Power banks come with everything built-in, but of course they cost more.

The Bees will draw about 250mA. So something with 2200mAh of capacity - a common minimum - should last about 9 hours, but everyone is making all of those numbers up. Battery specs are notoriously overstated and unreliable. We'd probably get more like 6-7 hours out of it. Still, not bad.

The Flowers will probably draw more. Whatever is powering them will also have to power anything else on that circuit. That's to say if the ESP is outputting a PWM to control a beefy motor swinging a sledge hammer into a gong, the power supply will also need to be able to handle the power draw of that beefy motor.

I'm going to look into vape batteries as a potential option for the Bees. The internet is usually pretty cautionary about such things, but I'm not planning on doing any of the sketch stuff, so we'll see.

The Flowers will probably need something more significant, but it really depends on what we're trying to do.